Session 01 — GNS3 Installation (macOS)

> This guide covers **only the macOS-specific installation of GNS3**. Once GNS3 is running, follow the Ubuntu guide (`01_gns3_setup.md`) from **Step 3** onward for everything else: first launch, templates, topology, network config, and verification.

Credentials summary (keep handy):

Machine	Username	Password
Alice / Bob	`ubuntu`	`ubuntu`
Darth	`pi`	`raspberry`
Cisco R1 / SW1	—	no login required

Step 1 — Install Homebrew

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.s
```

Add Homebrew to your PATH (shown at the end of the install output):

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Step 2 — Install dependencies and GNS3

```
brew install qemu cmake libpcap libelf pkg-config pipx
brew install --cask gns3
```

`brew install --cask gns3` installs both the GUI (`.app`) and a bundled server. The bundled server has Python dependency conflicts on macOS — **do not use it**. The next step installs the correct server.

Step 3 — Install GNS3 server via pipx

```
pipx install gns3-server
```

Verify:

```
~/.local/bin/gns3server --version
```

> Always launch the server with `~/.local/bin/gns3server`, not the one bundled in `/Applications/GNS3.app`.

Step 4 — Build and install ubridge from source

`ubridge` is not available in Homebrew on macOS.

First, make sure the Xcode Command Line Tools are installed (required for `git` and `make`):

```
xcode-select --install
```

If already installed you will see an error — ignore it and continue.

```
cd /opt
sudo git clone https://github.com/GNS3/ubridge.git
sudo chown -R $(whoami) ubridge
cd ubridge
make
sudo make install
which ubridge   # should print /usr/local/bin/ubridge
```

Step 5 — Build and install dynamips from source

```
cd /opt
sudo git clone https://github.com/GNS3/dynamips.git
sudo chown -R $(whoami) dynamips
cd dynamips
mkdir build && cd build
cmake .. -DDYNAMIPS_CODE=stable
make
sudo make install
which dynamips  # should print /usr/local/bin/dynamips
```

Step 6 — Fix macOS TCP socket bug in gns3-server

Without this fix, console connections to devices will fail. The issue is that macOS uses `TCP_KEEPALIVE` where Linux uses `TCP_KEEPIDLE`.

```
TELNET_FILE=$(find ~/.local/pipx/venvs/gns3-server -name 'telnet_server.py')
sed -i '' 's/TCP_KEEPIDLE/TCP_KEEPALIVE/g' "$TELNET_FILE"
grep -n 'TCP_KEEP' "$TELNET_FILE"   # verify the fix
```

> **Important**: this fix must be reapplied every time you upgrade gns3-server (`pipx upgrade gns3-server`), because the upgrade replaces the file.

Step 7 — Start the GNS3 server

Open a dedicated terminal window and run:

```
~/.local/bin/gns3server
```

Wait until you see this line in the output before launching the GUI:

```
INFO  Starting server on 0.0.0.0:3080
```

Leave this terminal open. Then launch the GUI:

```
open /Applications/GNS3.app
```

In the Setup Wizard:

1. Select "Run the topologies on my local computer":

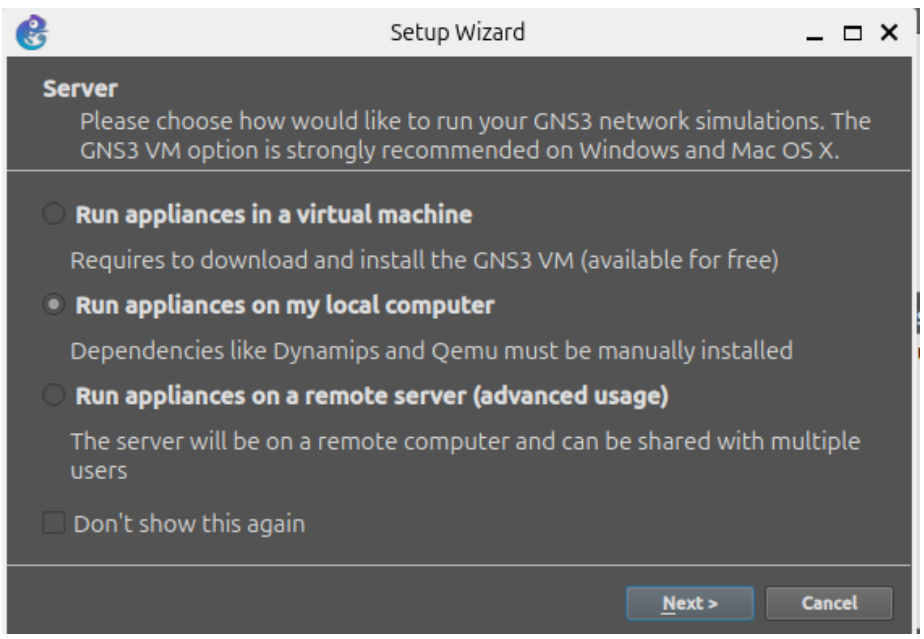

Figure 1: Setup Wizard — server type

2. Local server config — set path to `~/.local/bin/gns3server`, host `localhost`, port `3080`:

> The screenshot shows `/usr/bin/gns3server` (Ubuntu path) — on macOS use `~/.local/bin/gns3server` instead.

3. Connection successful:

4. Summary — verify settings:

5. GNS3 main window:

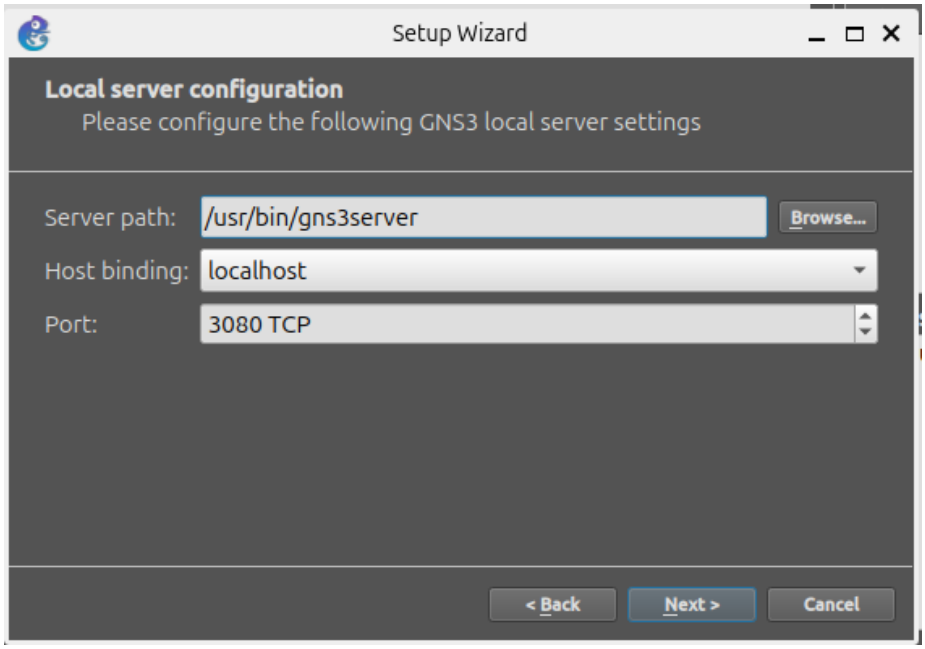

Figure 2: Setup Wizard — local server config

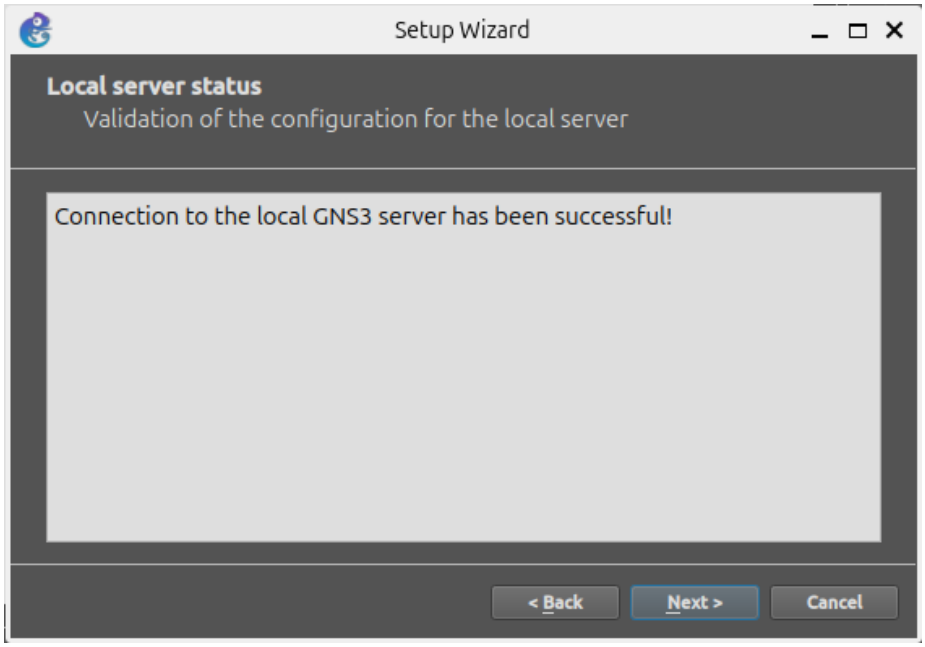

Figure 3: Setup Wizard — connection OK

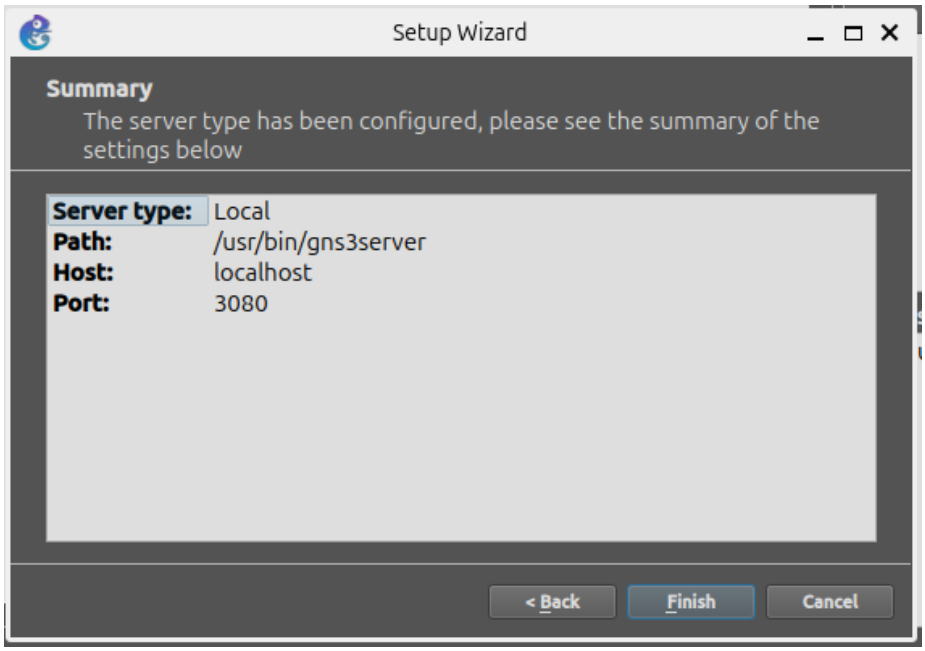

Figure 4: Setup Wizard — summary

6. Configure VNC viewer:

Go to **Edit** → **Preferences** → **General** → **Console applications** and set:

- **VNC viewer**: `vncviewer %h:%p`

  The macOS built-in Screen Sharing viewer does not work correctly with GNS3. Install TigerVNC instead:

  ```
  brew install --cask tigervnc-viewer
  ```

  The binary is `vncviewer` — same command as on Linux.

Step 8 — Configure NAT (internet access for the lab)

macOS does not have `virbr0` or `libvirt`. The equivalent is built manually using a virtual bridge (`bridge9`), a **fake ethernet pair** (`feth0`/`feth1` — the macOS equivalent of Linux `veth`), and `pfctl` for NAT.

> These settings are **not persistent** — they are lost on reboot. Run the `start_nat_mac.sh` script every time before opening GNS3.

1. Create the fake ethernet pair and attach it to the bridge:

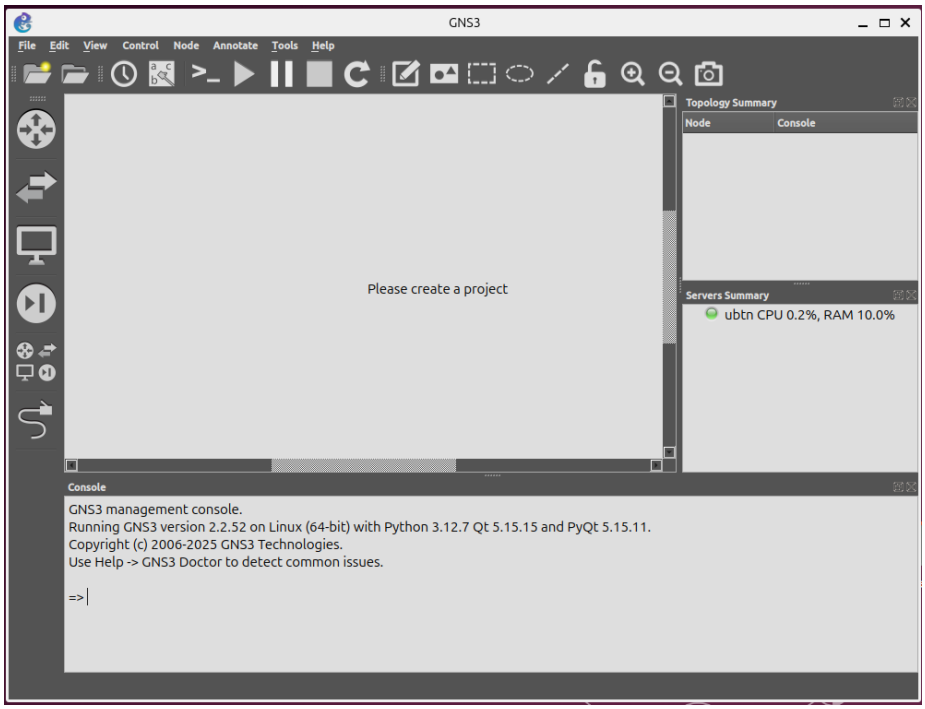

Figure 5: GNS3 main window

```
sudo ifconfig feth0 create 2>/dev/null || true
sudo ifconfig feth1 create 2>/dev/null || true
sudo ifconfig feth0 peer feth1 2>/dev/null || true   # "Resource busy" on re-run is fine
sudo ifconfig feth0 up
# feth1 needs a dummy IP - GNS3 refuses interfaces without a netmask
sudo ifconfig feth1 192.168.122.254 netmask 255.255.255.0 up
```

2. Create the bridge, assign it the gateway IP, and add feth0 as member:

```
sudo ifconfig bridge9 create 2>/dev/null || true
sudo ifconfig bridge9 192.168.122.1 netmask 255.255.255.0 up
sudo ifconfig bridge9 addm feth0 2>/dev/null || true   # already a member on re-run is fine
```

Verify the bridge is active:

```
ifconfig bridge9 | grep status
# Expected: status: active
```

3. Fix the route so it points to bridge9 (not feth1):

```
sudo route delete -net 192.168.122.0/24 2>/dev/null || true
sudo route add -net 192.168.122.0/24 -interface bridge9
```

4. Enable IP forwarding:

```
sudo sysctl -w net.inet.ip.forwarding=1
sudo sysctl -w net.inet.icmp.bmcastecho=1
```

5. Detect your internet interface and load the NAT rule:

```
WAN_IF=$(route get default | awk '/interface:/ {print $2}')
echo "Internet interface: $WAN_IF"   # usually en0 (Wi-Fi) or en1 (Ethernet)
```

6. Load the NAT rule with pf:

```
sudo pfctl -F all 2>/dev/null || true
echo "nat on $WAN_IF from 192.168.122.0/24 to any -> ($WAN_IF)" | sudo pfctl -f - -e 2>/dev/
```

Verify the rule is loaded:

```
sudo pfctl -s nat
# Expected: nat on en0 inet from 192.168.122.0/24 to any -> (en0) round-robin
```

7. Configure the Cloud node in GNS3:

- Add a **Cloud** node to the canvas
- Right-click → **Configure** → check **Show special ethernet interfaces**
- Select `feth1` (not `bridge9`, not `virbr0`) → **Add** → **OK**
- Connect the Cloud node to the router's external interface (`FastEthernet0/0`)

  The traffic path is: GNS3 → `feth1` → `feth0` → `bridge9` → `pf NAT` → `en0` → Internet. `feth1` is the GNS3-facing side of the pair; `feth0` is the bridge-facing side.

> **Do NOT assign `bridge9` to the Cloud node** — traffic must enter via `feth1` and cross the feth pair to reach the bridge. Connecting GNS3 directly to `bridge9` will result in a non-functional setup.

All of the above steps are automated in `start_nat_mac.sh`.

Continue with the Ubuntu guide

From this point, **follow `01_gns3_setup.md` from Step 3** — first launch wizard, disk images, device templates, topology, VM configuration, and verification are identical.

The only difference: in the Cloud node configuration, select **`feth1`** instead of `virbr0`.

Troubleshooting (macOS-specific)

Console connection to devices fails (blank terminal)

The TCP_KEEPALIVE fix (Step 6) was not applied, or gns3-server was already running when you applied it. Stop the server, reapply, restart.

bridge9 / feth0 disappear after reboot

Expected — macOS does not persist virtual interfaces. Rerun Step 8 before every session.

GNS3 GUI cannot connect to server

Make sure `~/.local/bin/gns3server` is running in a separate terminal before launching the GUI.

dynamips or ubridge not found by GNS3

Go to **Edit → Preferences → Dynamips** and **Edit → Preferences → General** and set the paths to `/usr/local/bin/dynamips` and `/usr/local/bin/ubridge`.

QEMU VMs do not start

```
which qemu-system-x86_64   # must return a path
brew reinstall qemu         # if missing
```